

Appendix D: Conversational Forking: Non-Linear Prompt Engineering

The development of the Origin Data Management System through the Elaborated Action Design Research methodology revealed an unexpected but significant auxiliary contribution: the emergence of a novel method for steering Large Language Model outputs in research and development contexts. This chapter presents the Conversational Forking (CF) method, a non-linear approach to prompt engineering that fundamentally reimagines how researchers and practitioners interact with LLMs for complex, iterative problem-solving tasks.

The CF framework emerged organically from interactions with LLMs throughout the eADR cycles, representing what Mullarkey and Hevner (2019) describe as the “emergent learning” characteristic of action design research. Rather than treating LLM conversations as linear, immutable sequences, the method re-conceptualizes them as dynamic, branchable structures that can be explored, revised, and optimized through strategic navigation. This approach addresses critical gaps in existing prompt engineering methodologies, particularly for research and development contexts where iterative refinement and complex problem-solving are paramount.

Background and Related Work

Evolution of Prompt Engineering

Prompt engineering has evolved from simple query formulation to sophisticated frameworks designed to elicit specific behaviors from LLMs. Early approaches focused on task-specific templates (Hong et al., 2023), while recent work has developed comprehensive frameworks for structured interaction (Thakur, 2024). The field has rapidly matured from ad hoc experimentation to systematic methodologies. Yet, significant gaps remain in addressing the iterative, exploratory nature of research and development work.

The Chain of Thought (CoT) prompting introduced by Wei et al. (2022) fundamentally changed how we approach complex problem-solving with LLMs by encouraging step-by-step reasoning. This was followed by frameworks like Tree of Thought (Yao, Yu, et al., 2023), which explored multiple reasoning paths simultaneously. However, these approaches maintain a fundamentally linear conversation structure, even when exploring various solution paths. They assume that each interaction builds sequentially on the previous one, without provision for revisiting, revising, or exploring alternative branches from earlier decision points.

Recent developments have introduced the distinction between prompt engineering and context engineering. As Karpathy (2025) notes, context engineering focuses on providing LLMs with sufficient contextual information to complete tasks effectively, while prompt engineering concentrates on optimizing the structure and phrasing of requests. The Conversational Forking method bridges these approaches by treating both prompt and context as mutable elements that can be strategically revised through non-linear navigation.

Current Framework Landscape

The contemporary prompt engineering landscape encompasses numerous frameworks, each with distinct approaches to structuring LLM interactions. Table D1 presents a categorized overview of prominent frameworks, highlighting the diversity of current approaches while revealing the gap that Conversational Forking addresses.

Table D1: Overview of Prompt Engineering Frameworks

Name	Components	Category
ACE	Audience, Context, Execution	Structured
APE	Active Prompt Engineering	Structured
BAB	Before, After, Bridge	Structured
CARE	Context, Action, Result, Example	Structured
Chain of Thought	Step-by-step + Reasoning	Structured + Reasoning
CLEAR	Context, Language, Examples, Audience, Request	Structured
ReACT	Reasoning + Acting	Reasoning
RISE	Role, Input, Steps, Expectation	Role + Structured
RTF	Role, Task, Format	Role + Structured
Step-Back	Abstraction + Reasoning	Reasoning
Stepwise	Step-by-step prompt	Reasoning
TAG	Task, Action, Goal	Structured
Tree of Thought	Explore Multiple Reasoning Paths	Structured + Reasoning

This is a compilation of various prompt Frameworks (Abbas, 2023; Lo, 2023; Lynch et al., 2025; Patel, 2025; Qian et al., 2024; Sun et al., 2024; Thakur, 2024; Yao, Zhao, et al., 2023; Zheng et al., 2023)

These frameworks can be categorized into several paradigms. Structured approaches like CLEAR (Lo, 2023) and CARE (Abbas, 2023) offer systematic frameworks for prompt construction, emphasizing clarity and logical flow while assuming single-turn or simple multi-turn interactions. Reasoning-based approaches such as ReAct (Yao, Zhao, et al., 2023) and Step-Back prompting (Zheng et al., 2023) focus on enhancing LLM reasoning capabilities but don't

address the versioning and branching needs of iterative development. Role-based approaches like RISE and RTF leverage role-playing to improve performance in specific domains (Thakur, 2024) but lack mechanisms for systematic iteration and refinement.

Analysis of existing frameworks reveals critical limitations when applied to complex, iterative development tasks: (1) Linear conversation assumption—most frameworks assume forward-moving conversations where each interaction builds sequentially; (2) Context window constraints—no existing framework explicitly addresses managing conversations that exceed typical context windows; (3) Limited error recovery, current approaches lack steering mechanisms for recovering from suboptimal outputs; and (4) Single-instance focus, existing frameworks don’t account for distributed problem-solving across multiple LLM instances.

The Conversational Forking Method

Formal Definition

Conversational Forking (CF) is a non-linear context engineering method that treats LLM conversations as version-controlled, branchable structures rather than immutable linear sequences. The process enables users to: (1) Create “save points” in conversations at critical junctures; (2) Return to previous conversation states when current paths prove unproductive; (3) Explore multiple solution branches in parallel; (4) Merge insights from different conversation branches; and (5) Maintain conversation history as a reusable knowledge artifact.

This approach fundamentally reimagines the nature of human-LLM interaction, moving from a paradigm of sequential dialogue to one of exploratory navigation through solution spaces. Unlike traditional prompt engineering that focuses on optimizing individual prompts, CF optimizes entire conversation trajectories through strategic branching and pruning.

Core Components

The CF method consists of four primary components that work synergistically to enable non-linear problem-solving:

Conversation State Management

Conversation State Management treats each significant exchange as a potential branching point. Users maintain awareness of the conversation's structure, identifying moments where alternative approaches might yield better results. This requires a shift in mindset from viewing conversations as flowing rivers to seeing them as navigable networks of possibilities.

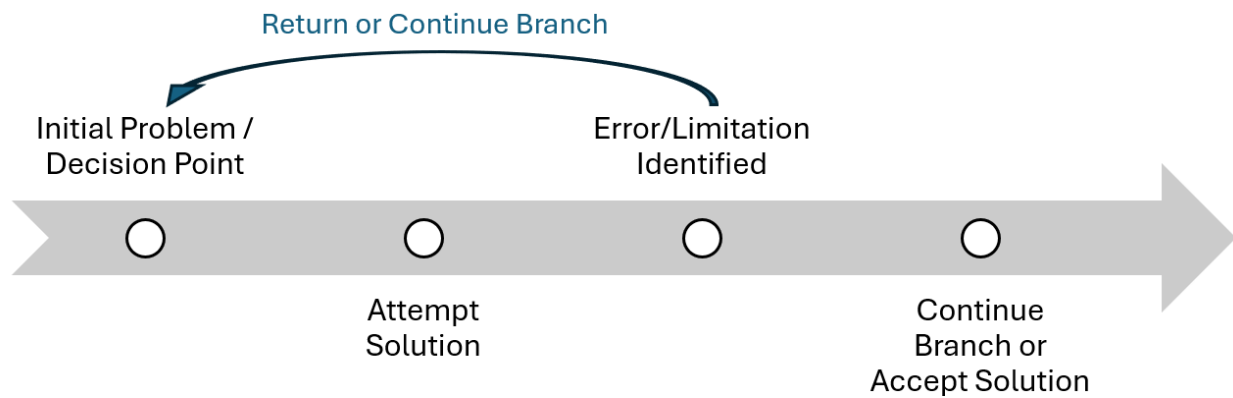


Figure D1: Prompt Workflow

Note. This figure displays the typical linear prompt workflow, including the option to roll back to a prior decision point. Strategic Forking Triggers

Strategic Forking Triggers

Strategic Forking Triggers identify specific scenarios that warrant conversation forking:

- (1) Error cascades, when an initial approach leads to compounding errors;
- (2) Context overflow, when conversation length approaches context limits;
- (3) Exploratory branching, when multiple viable approaches exist;
- and (4) Optimization opportunities, when functional but suboptimal solutions emerge.

A user is not informed of these triggers by an automated alert from the LLM; instead, they are situational cues that the practitioner learns to recognize as part of applying the Conversational Forking method. This process is analogous to how a software developer uses a version control system like Git. Git does not tell the developer when to create a new branch; the developer recognizes a strategic moment—such as the start of a new feature or a complex bug fix—and makes a deliberate decision to fork the codebase.

Similarly, a user applying this method develops an awareness of the conversation’s state and learns to identify these triggers:

- The user recognizes an Error Cascade when they notice the LLM repeatedly making the same logical mistake or generating flawed code based on a faulty premise.
- They anticipate Context Overflow when they observe the LLM “forgetting” instructions from earlier in the conversation.
- They identify a moment for Exploratory Branching when a problem presents multiple viable paths that are best investigated in parallel.
- They see an Optimization Opportunity when the LLM provides a working solution. Still, the user intuits that a more elegant or efficient solution is possible by reframing the prompt from an earlier point.

Branch Management Strategies

Branch Management Strategies employ three primary approaches. Divergent exploration creates multiple branches to explore fundamentally different approaches to the same problem.

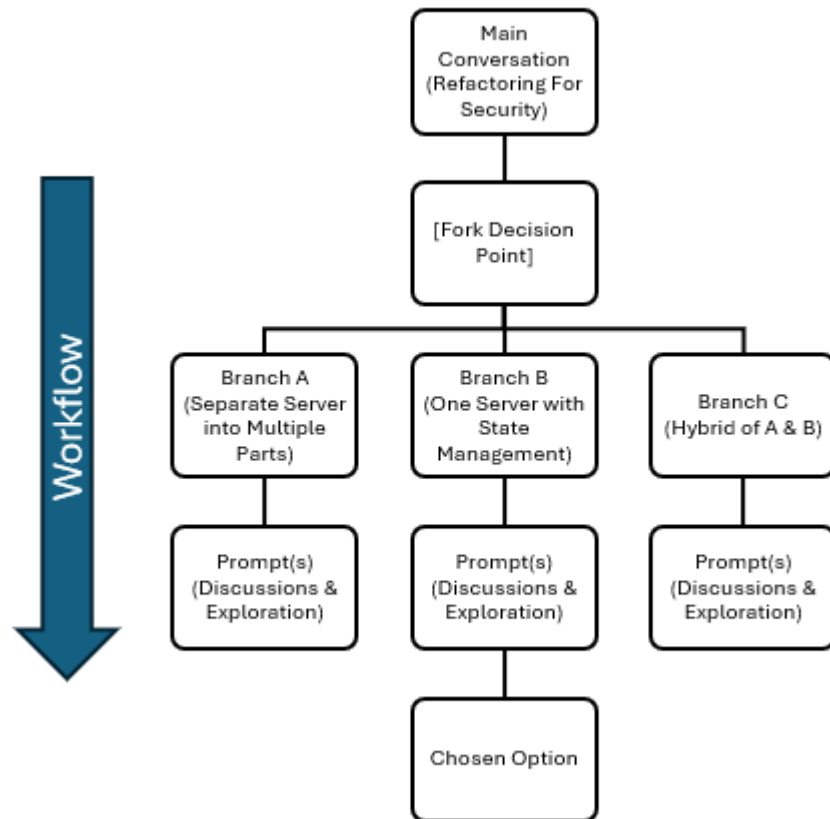


Figure D2: Example of Conversational Forking Divergent Exploration

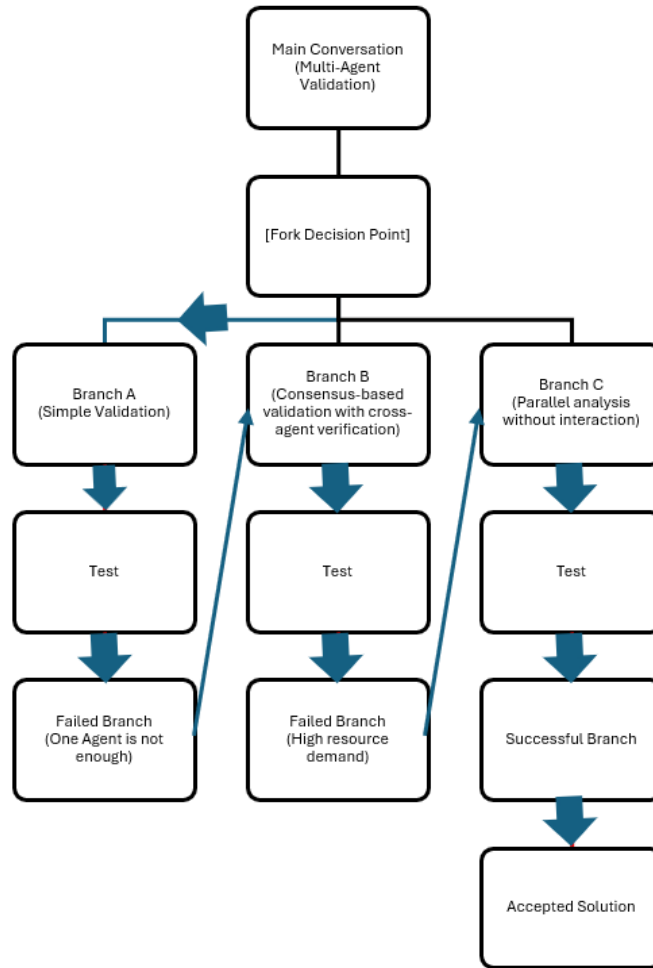


Figure D3: Example of Conversational Forking Convergent Refinement

Note. Convergent refinement uses insights from failed branches to inform new attempts, creating a learning loop across conversation instances.

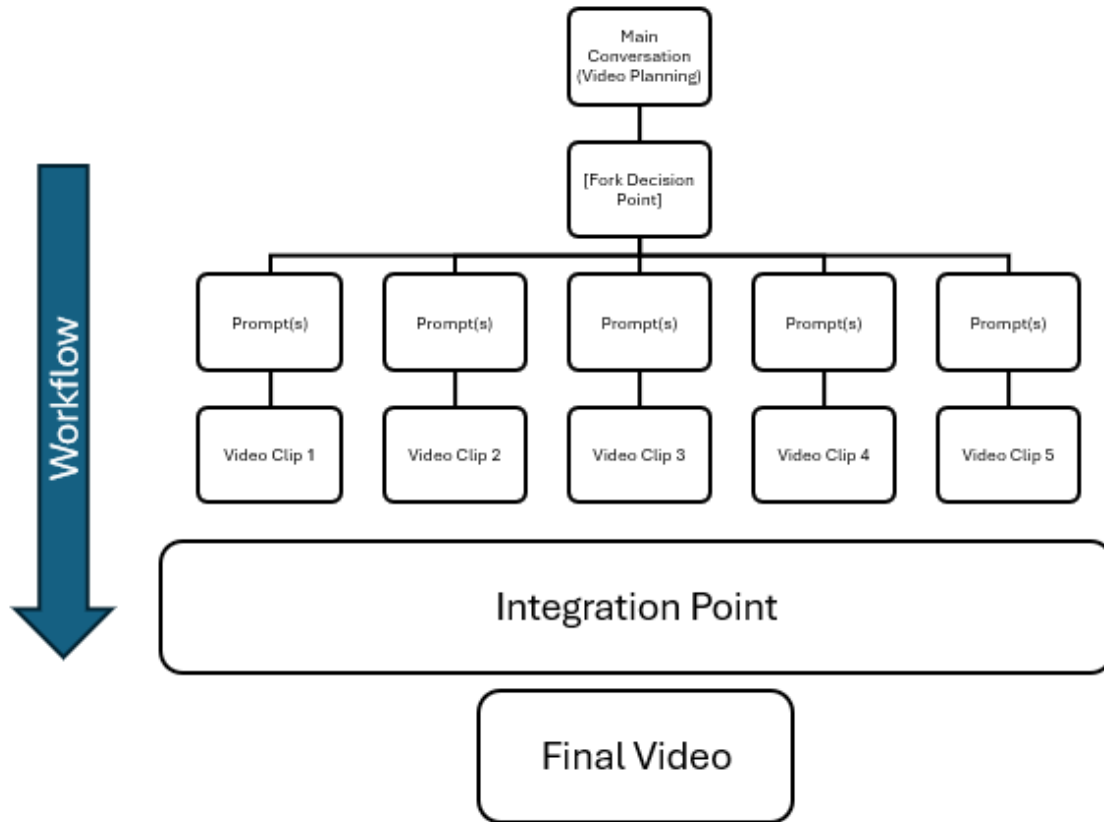


Figure D4: Example of Conversational Forking Parallel Processing

Note. Parallel processing distributes sub-problems across multiple conversation branches, then integrates results.

Context Preservation Mechanisms

Context Preservation Mechanisms maintain coherence across branches through explicit state summaries, which provide concise descriptions of current progress and constraints; artifact propagation, which carries forward code or insights across branches; and meta-prompting, which creates prompts specifically designed to initialize new branches with appropriate context.

Table D2: The Conversational Forking Process

Stage	Actions	Purpose
Initial Engagement	Establish the problem, context, and success criteria Identify potential areas of complexity.	Identify the problem and the output you want.
Progressive Development	Implement LLM suggestions or your own ideas Evaluate results, outcomes, and learnings.	Execute your prompt strategy.
Fork Decision Point	Assess current trajectory/state Determine branching strategy	Assess past, present, and possible future outputs.
Branch Creation	Craft a new initialization prompt Launch a new conversation branch.	Start a new branch or prune to a previous one.
Branch Integration	Compare outcomes across branches Select optimal path forward.	Assess branches for consolidation.
Implement Branch Strategy	Extract and implement reusable components Archive conversation branches.	Execute Conversational forking or move on to the next problem.